

Integración y pruebas

2. Integración y pruebas

Último apartado del módulo. Responde a la pregunta que da sentido a todo lo anterior: ¿cómo sabes que tus componentes, diseñados por separado, funcionan bien juntos?

Probar un componente aislado — repaso

Ya has construido este tipo de test en la Actividad 4.1, con el mismo patrón visto en la teoría de "Componentes":

```
@ExtendWith(MockitoExtension.class)
class LibroConsultaServiceImplTest {
    @Mock
    private LibroRepository libroRepository;
    @InjectMocks
    private LibroConsultaServiceImpl libroConsultaService;

    @Test
    void existeLibro_DebeDevolverTrue_CuandoExiste() {
        when(libroRepository.existsById(1L)).thenReturn(true);
        assertTrue(libroConsultaService.existeLibro(1L));
    }
}
```

Un test aislado con mocks prueba la lógica interna de un componente, sin ninguna base de datos real de por medio — rapidísimo, perfecto para verificar el comportamiento de una pieza concreta.

Un mock no miente, pero puede estar equivocado

Cuando escribes `when(libroRepository.existsById(1L)).thenReturn(true)`, ese valor lo has puesto tú a mano — el test no ha comprobado que `LibroRepository` se comporte así de verdad, solo que tu código reacciona bien si se comporta así.

Piensa en el borrado en cascada: al borrar un Libro, el catálogo publica un evento que `RabbitMQ` enruta hacia una cola y un listener asíncrono termina llamando a `deleteByLibroId` sobre `NotaLecturaRepository`. Un test aislado, con mocks, podría comprobar "si me llega el evento, borro las notas correctamente" — pero nunca demuestra que el evento llega.

Antes de seguir, predícelo

Tu proyecto tiene, sobre el mismo exchange, dos colas independientes escuchando el borrado de un videojuego: la del registro de actividad (todas las videojuego.*) y la del borrado en cascada de reseñas (solo videojuego.eliminado). Imagina que, sin querer, dejas la segunda cola enlazada a la routing key equivocada — videojuego.actualizado en vez de videojuego.eliminado. ¿Lo detectaría un test aislado (con mocks)? ¿Y un test de integración con RabbitMQ real?

Razona tu respuesta antes de seguir — la solución está en la siguiente diapositiva.

Comprueba tu respuesta

El test aislado NO lo detectaría — sigue probando "si me llega el evento, borro bien", y eres tú quien decide a mano que le llega. El test de integración SÍ: publicas el evento de borrado de verdad, y el listener nunca se dispara, porque su binding real no coincide con la routing key que se publica. Es exactamente el tipo de error de configuración que solo un RabbitMQ real puede delatar.

El test de integración usa infraestructuras reales

El test de integración de este flujo utiliza las infraestructuras reales que participan en lo que quiere comprobar, levantadas temporalmente con Testcontainers. Sobre el ejemplo de Libro, eso significa tres motores a la vez: PostgreSQL (catálogo), MongoDB (notas de lectura) y RabbitMQ (la cadena de eventos):

```
@Testcontainers
class LibroFlujoCompletoIntegrationTest {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine");
    @Container @ServiceConnection
    static MongoDBContainer mongodb = new MongoDBContainer("mongo:8");
    @Container @ServiceConnection
    static RabbitMQContainer rabbitmq = new RabbitMQContainer("rabbitmq:4-management");
    // ...
}
```

Por qué Redis se queda fuera

Redis no aparece en este test porque la caché está deshabilitada en el perfil test, como decidiste en PSP: aquí no estás comprobando el comportamiento de la caché, sino la integración catálogo → mensajería → reseñas.

Los tres contenedores son los servicios reales del docker-compose.yml de este proyecto de ejemplo, arrancados solo para este test. Este es el ejemplo definitivo de "probar la integración de componentes reales, no mockeados": que el catálogo, las notas de lectura y el borrado en cascada — contruidos por separado, en temas distintos — funcionen correctamente juntos, en la aplicación completa.

En tu proyecto no partes de cero

No vas a crear una clase nueva desde cero en la Actividad 4.3: tu `VideojuegoApiIntegrationTest` (Actividad 2.3) ya levanta PostgreSQL desde el principio, y ya has ido añadiendo MongoDB y RabbitMQ a esa misma clase en actividades posteriores. En la 4.3 amplías ese test con el flujo que todavía falta (reseñas y borrado en cascada), no lo repites.

La pirámide de tests

No es casualidad que tu proyecto tenga muchos tests aislados y solo un puñado de tests de integración. Cuantos más motores reales levanta un test, más lento y más caro es de mantener, así que conviene tener pocos de esos y muchos del tipo barato y rápido:



Testcontainers · pocos · lentos · levantan Docker de verdad

VideojuegoApiIntegrationTest



Mockito (aislados) · muchos · rapidísimos · sin BD real

CatalogoConsultaServ
icImplTest

ReviewsConsultaServi
celImplTest

...

...

...

Un proyecto sano se parece a esta pirámide

Base ancha de tests aislados que revisan cada pieza suelta en milisegundos, y una punta estrecha de tests de integración que revisan, más despacio, que esas piezas encajan de verdad.

Si lo hicieras al revés —todo con Testcontainers, nada con mocks— cada push tardaría minutos en verificarse y encontrar un fallo concreto sería más lento, porque un test de integración que falla no te dice qué pieza exacta ha fallado, solo que algo en la cadena no encaja.

Cuándo conviene cada tipo de test

	Test aislado (mocks)	Test de integración (Testcontainers)
Qué prueba	La lógica interna de un componente	Que varios componentes reales trabajan bien juntos
Velocidad	Muy rápido	Más lento (levanta contenedores reales)
Cuándo usarlo	Validaciones, cálculos de una pieza concreta	Flujos completos, entre módulos y motores distintos
Si falla, ¿qué te dice?	Exactamente qué pieza y qué caso concreto	Que algo en la cadena no encaja — toca investigar cuál

Ninguno sustituye al otro — un proyecto real necesita ambos niveles, en esa proporción: muchos aislados, pocos de integración.

"Documentar" un componente

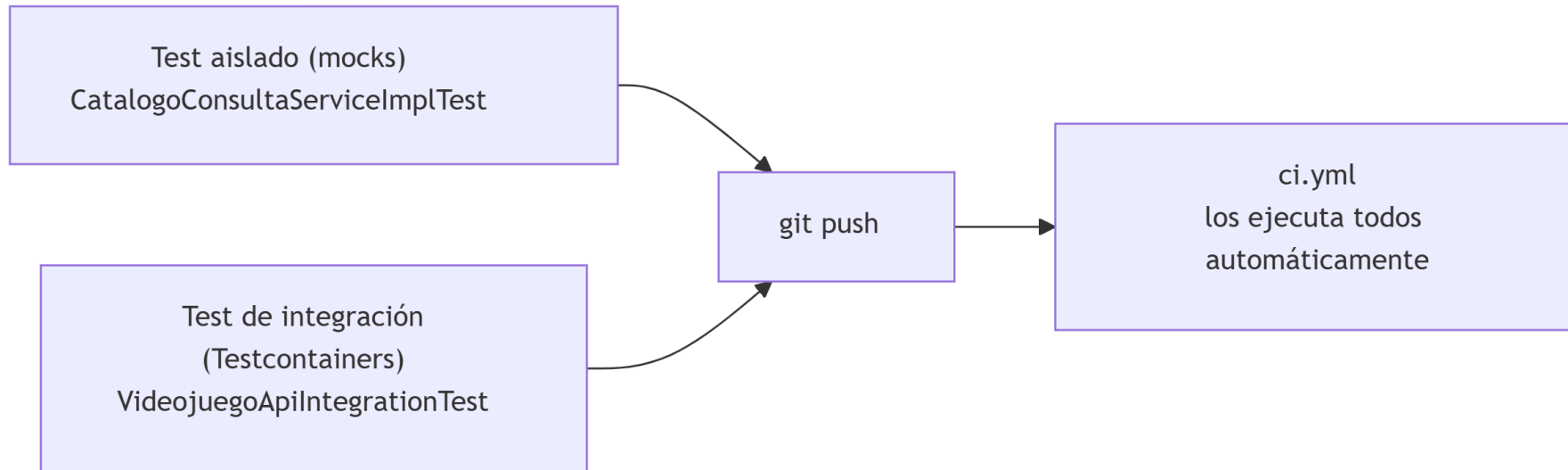
Ya has visto en el Tema 2 que un test bien nombrado (`existeLibro_DebeDevolverTrue_CuandoExiste`) es documentación viva: deja constancia de qué comportamiento se espera, y se actualiza solo — si el comportamiento cambia, el test falla y te avisa. Vale igual para cualquier tipo de test: cada test aislado documenta su propio componente por separado, y el test de integración de la Actividad 4.3 documenta cómo encajan todos juntos.

El círculo se cierra: CI

Tener los dos niveles de test escritos no basta si nadie los ejecuta en el momento justo — un test que solo corres cuando te acuerdas es, tarde o temprano, un test que no corres el día que más falta te hace. La solución no es disciplina personal, es automatización.

El concepto de integración continua lo has visto al principio del módulo (Tema 0); el `.github/workflows/ci.yml` de tu propio GameVault lo has construido después, en la Actividad 1.3 de PSP. Ese workflow ejecuta tus tests automáticamente en cada push — sin que tengas que acordarte de lanzarlos tú a mano cada vez.

De git push a ci.yml



Ahora mismo ese workflow solo ejecuta los tests que terminan en `ControllerTest` — un filtro que se queda corto en cuanto construyes tests de otro tipo, como los aislados y de integración de este tema.

En la Actividad 4.3

En la Actividad 4.3 eliminas ese filtro y dejas que Maven descubra automáticamente los tests que siguen sus convenciones habituales de nombre — el proyecto entero, con sus tests aislados y de integración, queda verificado en cada push.

Ideas clave

- Un test aislado (mocks) prueba la lógica interna de un componente; un test de integración (Testcontainers) prueba que varios componentes reales funcionan bien juntos.
- Un test de integración utiliza las infraestructuras reales que participan en el flujo: PostgreSQL, MongoDB y RabbitMQ. Redis queda fuera porque la caché está deshabilitada en el perfil test.
- La pirámide de tests: muchos aislados (rápidos, baratos) en la base, pocos de integración (lentos, más caros) en la punta — ninguno sustituye al otro.
- Un test bien nombrado es documentación viva: vale igual para un test aislado (documenta una pieza) que para uno de integración (documenta cómo encajan varias).
- El ci.yml del GameVault ejecuta tests automáticamente en cada push; en la Actividad 4.3 eliminas el filtro limitado a *ControllerTest para que Maven descubra también los aislados y de integración.